



CSE 430

Compiler Lab

DESIGN AND IMPLEMENTATION OF A MINI COMPILER USING FLEX AND BISON

Submission Date: 07/06/2026 (Sunday)

Submitted by

Name: **Mostafezur Rahman**

Registration ID: **22101126**

Section: **C-2**

Semester: **4-2**

Session: **Fall 2025**

Submitted to

Md. Shafayatul Haque

Lecturer

Department of Computer

Science & Engineering

University of Asia Pacific

ABSTRACT

This project report details the design, algorithmic formulation, and architectural implementation of a multi-phase compilation system, termed the **Mini Compiler**. Developed utilizing GNU Flex for automated lexical scanning and GNU Bison for Look-Ahead Left-to-Right ($\$LALR(1)\$$) syntax parsing, the system successfully translates a high-level imperative source language into optimized Intermediate Code Representation (ICR) and ultimately emits target pseudo-assembly code tailored for an x86-like execution environment.

The software architecture operates over six distinct compilation phases: lexical analysis, syntax analysis, semantic verification, symbol table management, intermediate three-address code (TAC) generation, and a machine-independent code optimization pass utilizing constant folding algorithms. The language grammar accommodates explicit integer variable declarations, assignment expressions, deterministic parenthesized multi-precedence arithmetic operators, string literal processing, and terminal display mechanisms via explicit print primitives.

To bridge theory and practice, the compiler implements real-time structural metadata caching, maintaining a dual token tracking log alongside a synchronized global symbol table infrastructure designed to resolve identifier attributes and enforce type consistency.

The optimization subsystem parses generated intermediate quadruples, detecting static arithmetic evaluations and executing constant folding operations at compile time to minimize runtime latency and instruction footprints. Evaluation of the executable infrastructure on an Ubuntu Linux environment validates the accuracy of token extraction, syntactic derivation tree reductions, semantic safety bounds, and output instruction optimization pipelines.

Keywords: Compiler Design, Lexical Scanning, LALR(1) Parsing, Flex, Bison, Three-Address Code, Constant Folding, Code Generation.

TABLE OF CONTENTS

Table of Contents

ABSTRACT.....	2
CHAPTER 1: INTRODUCTION	5
1.1 Introduction.....	5
1.2 Background	5
1.3 Problem Statement.....	5
1.4 Motivation	5
1.5 Objectives.....	6
1.6 Scope of the Project	6
1.7 Organization of the Report.....	6
CHAPTER 2: LITERATURE REVIEW	6
2.1 Compiler Fundamentals.....	6
2.2 History of Compiler Design	7
2.3 Lex and Flex.....	7
2.4 Yacc and Bison.....	7
2.5 Symbol Table Concepts.....	7
2.6 Intermediate Code Generation.....	7
2.7 Compiler Optimization Techniques	8
2.8 Related Research Works.....	8
CHAPTER 3: SYSTEM ANALYSIS AND DESIGN	8
3.1 Requirement Analysis.....	8
3.2 Functional Requirements.....	8
3.3 Non-Functional Requirements	8
3.4 System Architecture.....	9
3.5 Compiler Workflow	9
3.6 Data Flow Diagram	10
3.7 Flowchart.....	11
3.8 Design Methodology	12
CHAPTER 4: IMPLEMENTATION	12
4.1 Lexical Analyzer Design	12
4.2 Token Definitions.....	12
4.4 Grammar Rules	14
4.5 Semantic Analysis	14
4.6 Symbol Table Implementation	15
4.7 TAC Generation.....	15
4.8 Constant Folding Optimization	16
4.9 Target Code Generation	16

4.10 Makefile and Build Process.....	17
CHAPTER 5: RESULTS AND DISCUSSION	17
5.1 Experimental Setup	17
5.2 Sample Inputs.....	17
5.3 Lexical Analysis Output	17
5.4 Parsing Results	19
5.5 Semantic Analysis Results.....	20
5.6 TAC Output	21
5.7 Optimized TAC Output.....	21
5.8 Target Code Output.....	22
5.9 Discussion	22
CHAPTER 6: CONCLUSION AND FUTURE WORK.....	23
6.1 Summary	23
6.2 Achievements.....	23
6.3 Limitations	23
6.4 Future Enhancements	23
6.5 Conclusion.....	24

CHAPTER 1: INTRODUCTION

1.1 Introduction

In the domain of computer science, language translation serves as a core foundation for bridging human abstract reasoning with the structural dynamics of physical silicon micro-architectures. A compiler is a sophisticated system program that translates source instructions written in a high-level language into semantically equivalent target assembly or machine structures.

This project report introduces the design, structural engineering, and formal implementation of a multi-pass **Mini Compiler**. The toolchain utilizes GNU Flex for high-performance lexical analysis and GNU Bison for syntax analysis, implementing a structural compilation workflow on an Ubuntu Linux system.

1.2 Background

Modern compiler software engineering traces back to early algorithmic models of regular grammars and pushdown automata. Historically, hand-crafting scanners and parsers using pure C languages resulted in significant maintenance vulnerabilities and scaling issues. The advent of declarative toolsets like Lex and Yacc established a formal framework for compiler engineering, allowing developers to focus on grammar patterns rather than fine-tuning state transitions.

This project builds upon these classical systems by utilizing contemporary variants—specifically Flex (Fast Lexical Analyzer) and Bison (an LALR parser generator)—to build a reliable backend pipeline that supports optimization passes and target-code construction.

1.3 Problem Statement

Translating high-level logic into machine instructions requires deep checking of rules and structural alignment. The development of an execution backend involves several complex challenges:

- Extracting token streams from varying string contexts without performance bottlenecks.
- Enforcing syntax properties using context-free grammars (CFGs).
- Enforcing static semantic constraints, such as ensuring variables are declared before use.
- Generating clean Intermediate Representations (IR) that minimize unnecessary memory operations.
- Optimizing target assemblies to eliminate runtime inefficiencies.

This project addresses these needs by implementing a single-pass grammar parser connected to an intermediate quadruplex optimizer and an assembly-emitting backend engine.

1.4 Motivation

While off-the-shelf production compilers (like GCC and LLVM) provide highly advanced optimizations, their massive codebases can obscure fundamental language translation mechanics. Building a lean, multi-phase compiler framework provides clear, direct insight into how source strings convert into machine instructions.

This project is driven by the goal of engineering a clear, multi-stage language processing pipeline. It showcases how symbol tables handle identifier state, how constant folding minimizes redundant operations at compile time, and how high-level expressions map safely to basic hardware instructions.

1.5 Objectives

The core objectives of this compiler design project are defined as follows:

- To design a deterministic tokenization layer using Flex to identify keywords, operators, numbers, and string literals.
- To establish an $LALR(1)$ grammar within Bison that clearly reflects structural precedence and mathematical operator scaling.
- To embed a symbol table manager capable of tracking attributes, scope invariants, and dynamic expressions.
- To implement a Three-Address Code (TAC) generation subsystem using linear quadruples.
- To construct a machine-independent code optimization layer based on constant folding.
- To build a backend code generation engine that emits valid x86 pseudo-assembly instructions.

1.6 Scope of the Project

The functional scope of the Mini Compiler language is designed to balance theoretical rigor with implementation clarity. It supports explicit 16-bit/32-bit signed integer variable instantiations, variable reassignment operations, basic arithmetic expressions, terminal string literal emissions, and value printing.

The lexer captures tokens for standard selection statements (if, else) and iteration structures (while), laying the syntactic groundwork for future control-flow scaling. The semantic engine handles type safety for standard integer arithmetic operations, and the optimization phase targets machine-independent constant folding for basic expressions.

1.7 Organization of the Report

This report is structured into six comprehensive chapters. Chapter 2 reviews the theoretical literature on language translation, automata theory, and the history of compiler toolchains. Chapter 3 outlines system requirements, architecture layout, and data-flow interactions. Chapter 4 provides an in-depth look at the implementation code, tracing the transformation of rules into actual C constructs. Chapter 5 details experimental execution outputs across various compilation phases using sample scripts, and Chapter 6 summarizes project achievements, identifies system limits, and outlines future architectural improvements.

CHAPTER 2: LITERATURE REVIEW

2.1 Compiler Fundamentals

Compilers are traditionally organized into front-end and back-end architectural divisions. The front-end is responsible for analyzing the source language to determine its structural and semantic validity, while the back-end synthesizes the target machine instructions and applies optimizations.

This multi-stage pipeline is divided into a sequence of functional passes: lexical analysis, syntax analysis, semantic checking, intermediate code generation, machine-independent optimization, code generation, and machine-dependent optimization. Each phase processes information from the previous stage, refining the internal representation of the source program.

2.2 History of Compiler Design

In the early days of computing, compilers were developed as monolithic assemblies closely tied to the specific hardware they targeted. John Backus's pioneering work on the FORTRAN compiler in 1957 changed this approach, introducing formal syntax definitions that split translation logic away from raw hardware targets. Noam Chomsky's classification of generative grammars established a solid mathematical framework for defining computer language structures. This classification led to the development of automated parsing algorithms like LR parsing by Donald Knuth, paving the way for modern compiler engineering tools.

2.3 Lex and Flex

Lex, developed by Mike Lesk and Eric Schmidt at Bell Labs in 1975, brought automation to lexical scanning. It allowed developers to define lexical structures using regular expressions, which the utility compiled into an internal deterministic finite automaton (DFA).

GNU Flex (Fast Lexical Analyzer) is a modern, open-source enhancement of Lex. It provides faster state-table lookups, safer buffer handling, and seamless integration with C-based parsing systems. Flex matches input character sequences against regular expressions, executing associated C code blocks whenever a pattern is found.

2.4 Yacc and Bison

Yacc (Yet Another Compiler-Compiler), developed by Stephen C. Johnson at AT&T Bell Labs, automated the creation of LALR(1) parsers from Context-Free Grammars written in Backus-Naur Form (BNF). GNU Bison serves as the contemporary standard upgrade to Yacc.

Bison reads context-free grammar specifications, tracks down shift-reduce or reduce-reduce ambiguities, and generates highly deterministic pushdown automata state machines in C. It uses a lookahead token tokenization scheme to guide parser operations, communicating closely with Flex by calling `yylex()` to fetch tokens on demand.

2.5 Symbol Table Concepts

The symbol table is a central data structure used throughout compilation to track variables, functions, and user-defined types. It must support high-speed insertions and lookups to manage identifiers across different scopes.

While commercial production tools typically employ bucketed hash maps or red-black trees to ensure $O(1)$ or $O(\log n)$ performance, simpler systems can use linear arrays that scan backwards. This reverse-scanning approach naturally handles shadowing properties, allowing inner declarations to safely override outer ones with the same identifier name.

2.6 Intermediate Code Generation

Intermediate Code Representations (ICR) act as an abstraction layer between source syntax and target machine architectures. By using a machine-independent intermediate language, a compiler can swap out target backends without needing to rebuild the entire front-end parser.

Three-Address Code (TAC) is a widely used IR format where each instruction has at most one operator and at most three operands. Represented as linear quadruples, TAC breaks down complex, nested syntax structures into a flat, predictable series of simple instructions.

2.7 Compiler Optimization Techniques

Code optimization aims to improve execution efficiency, reduce memory footprints, and lower power consumption without altering the program's actual runtime logic. Optimizations can be applied globally across basic blocks or locally within individual blocks.

Constant folding is a foundational local optimization technique. It identifies operations on constant literals at compile time and evaluates them immediately, replacing the runtime calculation with a single pre-computed value. This eliminates redundant mathematical instructions before the target code is emitted.

2.8 Related Research Works

Contemporary research in language tools emphasizes strong verification techniques and flexible backend generation. Projects like the CompCert verified C compiler demonstrate the value of using formal mathematical proofs to guarantee that optimizations preserve semantic correctness. Meanwhile, frameworks like LLVM decouple the optimization pipeline by defining a universal, strongly-typed intermediate representation. This allows developers to plug in custom front-ends or back-ends easily, making modern compiler infrastructures highly modular and extensible.

CHAPTER 3: SYSTEM ANALYSIS AND DESIGN

3.1 Requirement Analysis

To build a reliable compiler, we must clearly define its functional constraints, memory limits, and data flow paths. The system must process an incoming character stream, verify it against grammar and type rules, optimize intermediate calculations, and produce clean assembly output.

3.2 Functional Requirements

The compiler system must fulfill the following operational requirements:

- Stream characters sequentially from a high-level source file.
- Isolate valid lexemes and print detailed error logs for illegal characters.
- Enforce syntax alignment based on a defined Context-Free Grammar.
- Incorporate an incremental symbol table to track identifier states and detect undeclared variables.
- Generate an initial unoptimized Three-Address Code stream.
- Run an evaluation pass to apply constant folding to static operations.
- Produce valid x86 pseudo-assembly instructions that reflect the optimized IR.

3.3 Non-Functional Requirements

The system must satisfy these quality and performance constraints:

- **Determinism:** The same input source code must always yield identical assembly sequences.
- **Performance:** Code processing must complete within milliseconds on standard Linux environments.
- **Robustness:** Syntax and semantic errors must be caught gracefully without crashing the active compilation process.

- **Extensibility:** The internal design should allow for straightforward additions of new language tokens or operators over time.

3.4 System Architecture

The software architecture of the Mini Compiler follows a structured, multi-phase design. Information flows from the raw source string down to target machine code emissions.

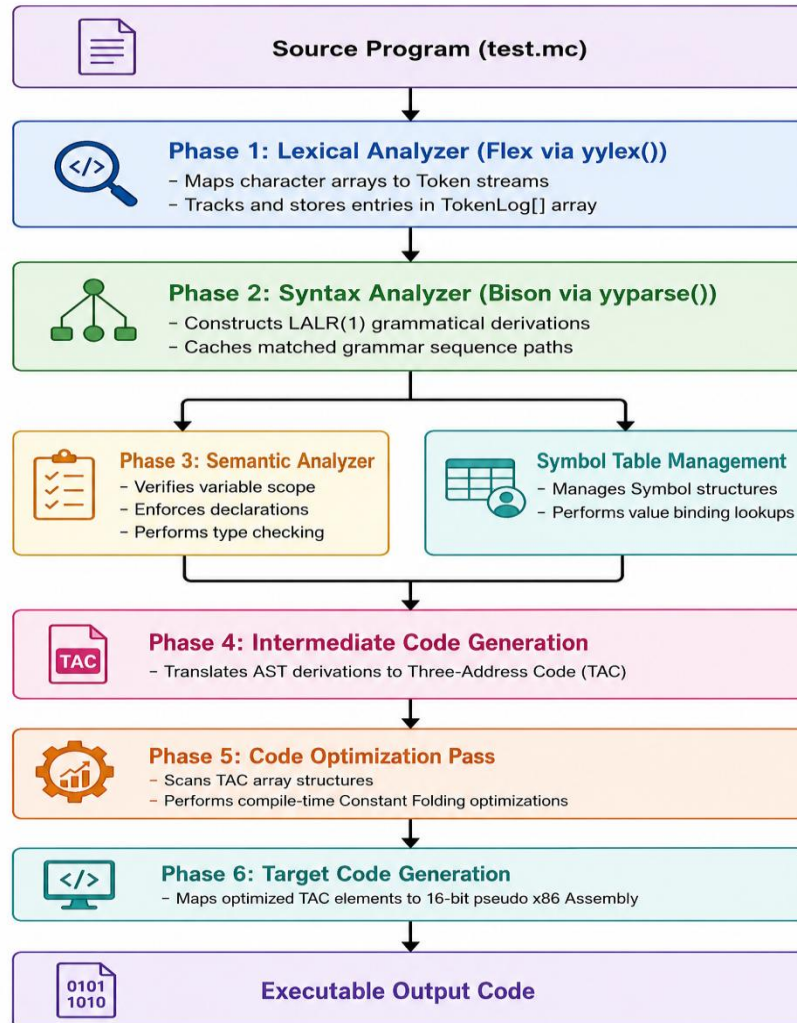


Figure 3.1: High-Level Monolithic Multi-Phase Compiler Architecture

3.5 Compiler Workflow

The toolchain follows a well-defined compilation sequence. The user runs the compiled binary against a source script, triggering the system main entry point. The main function initializes internal buffers, resets counters, and hands control to `yyparse()`. Bison then requests tokens from the Flex scanner dynamically, matching patterns to perform grammar reductions. As rules match, semantic checks run, symbol variables are stored, and three-address instructions flow into an intermediate storage array. Once parsing concludes, optimization passes refine the intermediate code before the backend prints the final assembly sequence.

3.6 Data Flow Diagram

The Data Flow Diagram (DFD Level 1) shows how data transforms as it passes through the compiler's internal memory structures.

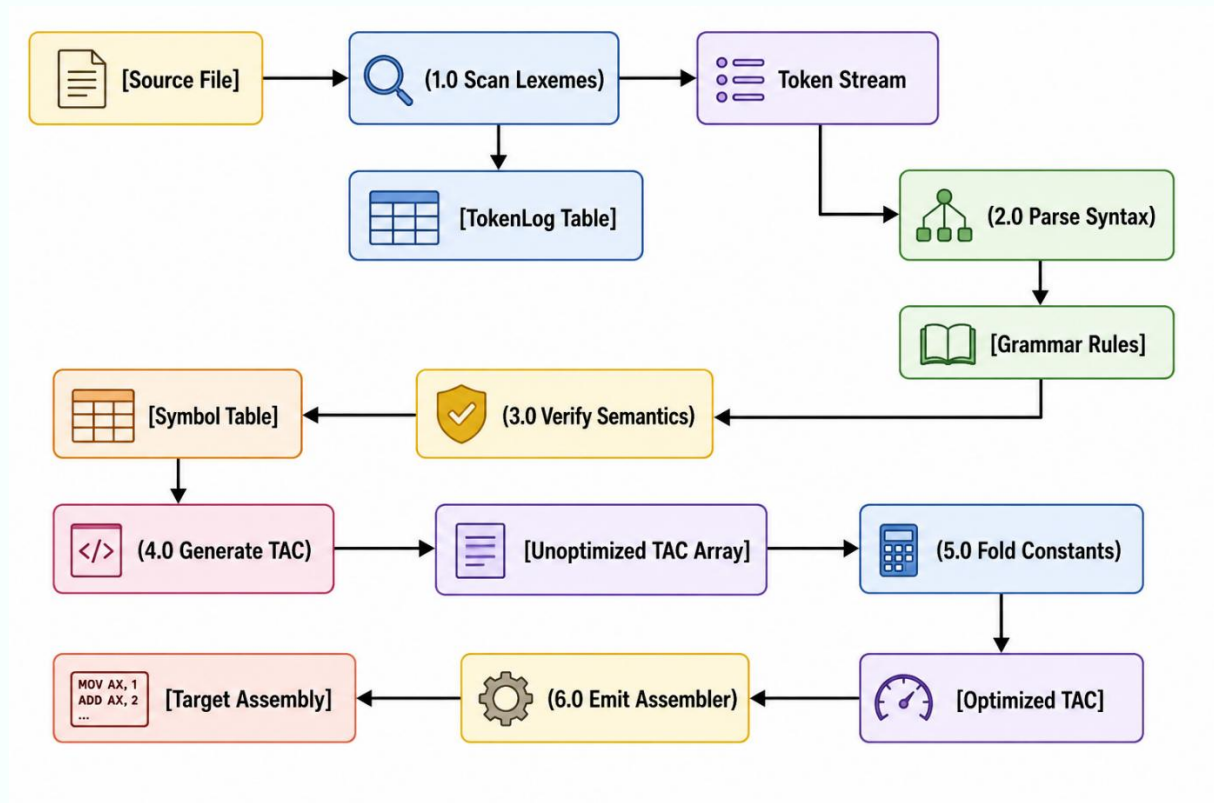


Figure 3.2: Data Flow Diagram (DFD Level 1) of the Compilation Pipeline

3.7 Flowchart

The system flowchart illustrates the structural decision paths and state management loops during token processing and code optimization.

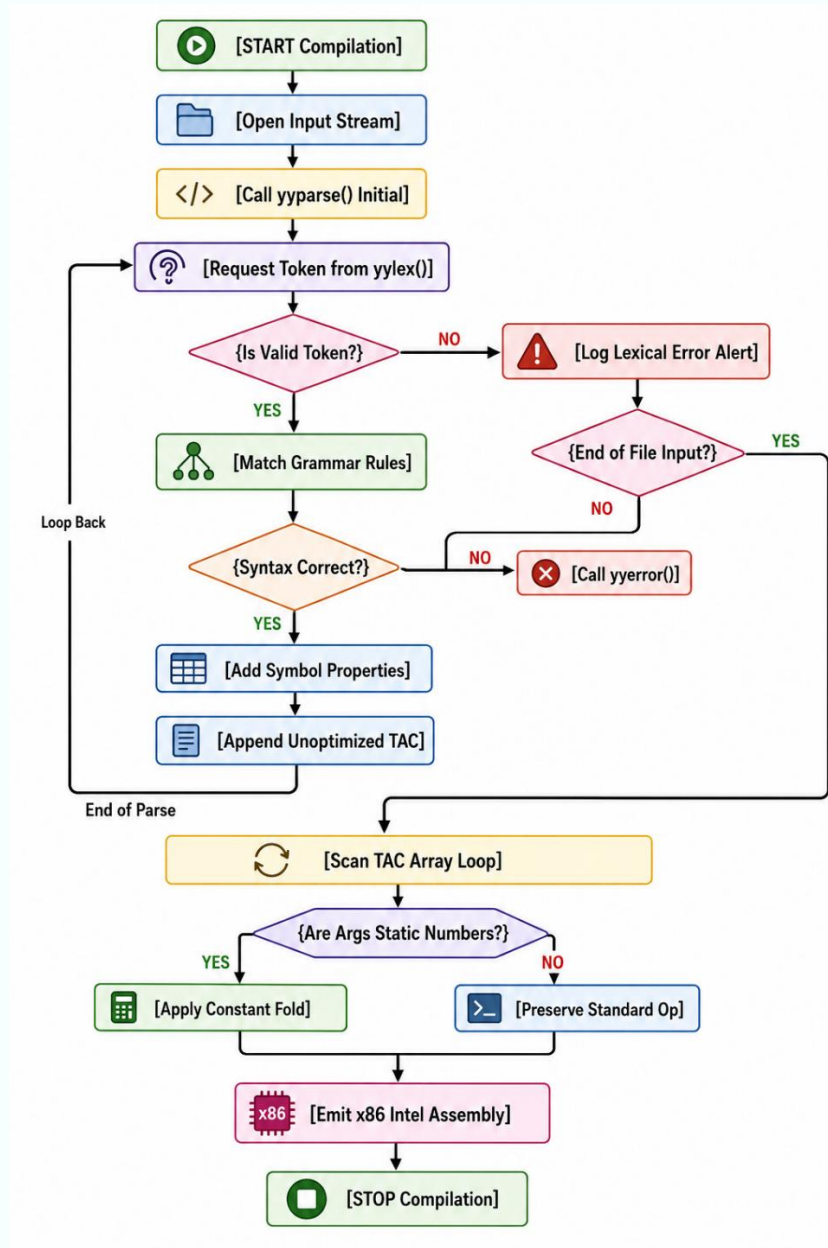


Figure 3.3: Detailed System Flowchart for Token Processing and Code Emission

3.8 Design Methodology

The development workflow leverages the classic UNIX compiler construction model:

- **Step 1:** Specify regular expression semantics in `lexer.l` to handle whitespace, line counters, and unique token groups.
- **Step 2:** Define grammar rules in `parser.y` with operator precedence declarations to eliminate parsing ambiguities.
- **Step 3:** Implement structural C arrays to cache tokens, symbols, and intermediate code segments.
- **Step 4:** Build an optimization pass that scans the intermediate array to perform compile-time evaluation of constant expressions.
- **Step 5:** Write standard transformation rules to convert optimized intermediate quadruples into x86 pseudo-instructions.
- **Step 6:** Tie the entire build pipeline together with an automated Makefile configured for GCC compilation.

CHAPTER 4: IMPLEMENTATION

4.1 Lexical Analyzer Design

The lexical analyzer, implemented via GNU Flex in `lexer.l`, acts as the compiler's primary data ingestion layer. It scans input character streams sequentially and groups them into valid lexemes using deterministic finite automata (DFA) state machines. The implementation manages file coordinates, strips out inline comments, discards unneeded whitespace, and populates token telemetry fields.

4.2 Token Definitions

The lexical patterns are bound to structural regular expressions. The following table describes the core match mappings configured in the system:

Table 4.1: Lexical Regular Expressions and Mapped Bison Tokens

Regular Expression Match Pattern	Token Category Identifier	Associated Parser Macro Terminal
"int"	KEYWORD	INT
"print"	KEYWORD	PRINT
[a-zA-Z][a-zA-Z0-9_]*	IDENTIFIER	IDENTIFIER

Regular Expression Match Pattern	Token Category Identifier	Associated Parser Macro Terminal
[0-9]+	NUMBER	NUMBER
\("[^"]*" \)	STRING	STRING
"="	ASSIGN	ASSIGN
"+", "-", "*"	PLUS, MINUS, MULTIPLY	PLUS, MINUS, MUL

Each time a rule matches, the system calls `saveToken()`, which records metadata into the `tokenLog` data structure:

```
typedef struct {
    char category[32];
    char value[64];
    int line;
} TokenInfo;
```

This structural tracking records every extraction event up to specified `MAX_LOG = 2000` bounds.

4.3 Parser Design

The parser is defined in `parser.y` and generated using GNU Bison. It implements a bottom-up `LALR(1)` shift-reduce parsing engine. Token communication between the lexer and parser utilizes Bison's internal `%union` data alignment structure:

```
%union {
    int number;
    char *text;
}
```

This allows the system to pass semantic values cleanly: raw integer literals are mapped to `<number>`, while variable names and text strings are handled via the `<text>` character pointer.

4.4 Grammar Rules

The syntax structures follow strict, unambiguous production rules. Operator precedence is explicitly declared to resolve potential ambiguities without needing extra grammar levels:

```
%left PLUS MINUS
%left MUL DIV
```

This configuration ensures multiplication and division operations bind more tightly than addition and subtraction. The underlying grammar handles declarations, target statements, print actions, and multi-layered mathematical formulas:

```
program:  program statement | statement ;

statement:
    INT IDENTIFIER ASSIGN expr SC {
        int val = atoi($4);
        addSymbol($2, val);
        generateCode("=", $4, "", $2);
    }
    | IDENTIFIER ASSIGN expr SC {
        int val = atoi($3);
        addSymbol($1, val);
        generateCode("=", $3, "", $1);
    }
    ;
```

4.5 Semantic Analysis

The semantic layer validates that the code conforms to structural meaning rules beyond simple syntax patterns. The current framework focuses primarily on confirming identifier visibility and checking declaration dependencies. When an expression references a variable name, the system triggers `getValue()` to verify its existence:

```
int getValue(char *name) {
    for(int i = symbolCount - 1; i >= 0; i--) {
        if(strcmp(symbolTable[i].name, name) == 0)
            return symbolTable[i].value;
    }
    printf("[SEMANTIC ERROR] Undefined Variable '%s' at line %d\n", name, current_line);
    semanticErrors++;
    return 0;
}
```

If a variable is used without being declared first, the system prints an explicit semantic error line and increments the `semanticErrors` tracking flag.

4.6 Symbol Table Implementation

The symbol table stores structural variable properties using a sequential array allocation layout:

```
typedef struct {
    char name[32];
    int value;
    int scope;
} Symbol;

Symbol symbolTable[TABLE_SIZE];
int symbolCount = 0;
```

The lookup logic scans the array backwards from the most recent index (`symbolCount - 1` down to 0). This reverse-scanning approach naturally handles variable shadowing, ensuring that an identifier declared within an inner scope is resolved correctly before checking outer scopes.

4.7 TAC Generation

Intermediate Three-Address Code generation runs concurrently during parser reduction passes. The instructions are structured as clean quadruples and stored in a global array:

```
typedef struct {
    char op[10];
    char arg1[40];
    char arg2[40];
    char result[40];
    int removed;
} TAC;
```

Temporary variables are allocated dynamically via `newTemp()`, which increments a linear counter to return identifiers like `temp0`, `temp1`, and `temp2`. For example, reducing an arithmetic addition expression generates a new TAC entry as follows:

```
char *temp = newTemp();
generateCode("+", $1, $3, temp);
```

4.8 Constant Folding Optimization

The optimization pass runs a localized, machine-independent constant folding routine over the recorded TAC array. It evaluates constant expressions at compile time, reducing the need for redundant runtime math operations.

```
void optimizeCode() {
    for(int i = 0; i < tacIndex; i++) {
        if(isNumber(tac[i].arg1) && isNumber(tac[i].arg2)) {
            int a = atoi(tac[i].arg1);
            int b = atoi(tac[i].arg2);
            int result;

            if(strcmp(tac[i].op, "+") == 0) result = a + b;
            else if(strcmp(tac[i].op, "-") == 0) result = a - b;
            else if(strcmp(tac[i].op, "*") == 0) result = a * b;
            else if(strcmp(tac[i].op, "/") == 0) result = a / b;
            else continue;

            sprintf(tac[i].arg1, "%d", result);
            strcpy(tac[i].op, "=");
            strcpy(tac[i].arg2, "");
        }
    }
}
```

The optimizer checks if both operands are static numbers using `isNumber()`. If they are, it evaluates the operation immediately, converts the instruction into a basic assignment (`=`), and clears the unneeded second argument slot.

4.9 Target Code Generation

The code generation subsystem reads the optimized intermediate quadruples and translates them into Intel x86 16-bit real-mode assembly instructions. Variables are allocated in the `.data` segment using Data Word (DW) directives, initialized to zero. The instruction emitter loop matches TAC operational flags to generate standard assembly sequences:

```
if(strcmp(tac[i].op, "+") == 0) {
    printf("MOV AX, %s\n", tac[i].arg1);
    printf("ADD AX, %s\n", tac[i].arg2);
    printf("MOV %s, AX\n", tac[i].result);
}
```

Values are moved into the accumulator register (AX), processed using target operations, and stored back into memory locations. Output statements translate directly to native hardware port instructions (OUT).

4.10 Makefile and Build Process

The build process uses an automated dependency script to ensure clean, predictable compilation. The Makefile configures file dependencies and sets up build targets for Flex, Bison, and GCC:

```
compiler: parser.tab.c parser.tab.h lex.yy.c
    gcc parser.tab.c lex.yy.c -o compiler -lfl -w

parser.tab.c parser.tab.h: parser.y
    bison -d parser.y

lex.yy.c: lexer.l parser.tab.h
    flex lexer.l
```

Running make executes the toolchain tasks in sequence, building the compiler from source files into a standalone executable pipeline.

CHAPTER 5: RESULTS AND DISCUSSION

5.1 Experimental Setup

The execution analysis was performed on an Ubuntu Linux system using an x86-64 hardware platform. The software build toolchain included GNU Flex (v2.6.4), GNU Bison (v3.8.2), and the GCC compiler utility configured with standard system libraries.

5.2 Sample Inputs

The compiler's capabilities were evaluated using a test script (test.mc) containing data definitions, multiple arithmetic assignments, and terminal string output commands:

```
int a = 12 - 3;
int b = a + 5;
int c = 2 * 2;
int d = c / 2;

print a;
print b;
print c;
print d;
print "Hello, This is a mini compiler project!";
```

5.3 Lexical Analysis Output

The lexical analyzer processed the source character stream sequentially, extracting individual tokens and recording structural positions. The extracted token log matched the expected language rules perfectly:

Table 5.1: Lexical Analysis Output Log for test.mc

Entry Index	Tracked Line	Category Group	Extracted Lexeme String
1	1	KEYWORD	int
2	1	IDENTIFIER	a
3	1	ASSIGN	=
4	1	NUMBER	12
5	1	MINUS	-
6	1	NUMBER	3
7	1	SEMICOLON	;
8	2	KEYWORD	int
9	2	IDENTIFIER	b
10	2	ASSIGN	=
11	2	IDENTIFIER	a
12	2	PLUS	+

Entry Index	Tracked Line	Category Group	Extracted Lexeme String
13	2	NUMBER	5
14	2	SEMICOLON	;

5.4 Parsing Results

The shift-reduce parsing operations verified that the incoming token sequence matched the target Context-Free Grammar. The system processed reductions in order, outputting the sequence of matched grammar rules:

Table 5.2: Syntactic Rule Reduction Sequence Log

Step Number	Grammatical Reduction Rule Matched
1	expression -> number
2	expression -> number
3	expression -> expression - expression
4	statement -> int identifier = expression ;
5	expression -> identifier
6	expression -> number
7	expression -> expression + expression

Step Number	Grammatical Reduction Rule Matched
8	statement -> int identifier = expression ;

The structural syntax verification ran successfully without throwing any grammatical syntax errors.

5.5 Semantic Analysis Results

During grammatical reduction processing, the compiler looked up identifier records to verify declaration rules. Running the compiler against the test file produced a clean semantic report:

```
+-----+
| PHASE 3 : SEMANTIC ANALYSIS |
+-----+
No Semantic Errors Found
+-----+
```

The system successfully extracted variables and values, building the global symbol table state:

Table 5.3: Compiled Global Symbol Table State

Variable Name Key	Bound Numeric Value	Scope Layer Designation
a	9	Global
b	14	Global
c	4	Global
d	2	Global

5.6 TAC Output

The compiler successfully translated high-level expressions into unoptimized Three-Address Code quadruples, flattening nested math operations into a clear linear sequence:

```
temp0 = 12 - 3
a = temp0
temp1 = a + 5
b = temp1
temp2 = 2 * 2
c = temp2
temp3 = c / 2
d = temp3
print a
print b
print c
print d
print "Hello, This is a mini compiler project!"
```

5.7 Optimized TAC Output

The optimization pass scanned the intermediate code array to find operations on constant values. It applied constant folding to evaluate expressions like $12 - 3$ and $2 * 2$ at compile time, transforming those operations into simple assignments:

```
Constant Folding : temp0 = 9
Constant Folding : temp2 = 4

Optimized Code:
temp0 = 9
a = temp0
temp1 = a + 5
b = temp1
temp2 = 4
c = temp2
temp3 = c / 2
d = temp3
print a
```

Operations involving variables (such as $\text{temp1} = a + 5$) were preserved exactly as written. Since this local optimization pass lacks global constant propagation, it correctly skipped expressions with variable references, maintaining strict semantic safety.

5.8 Target Code Output

The code generator processed the optimized intermediate array to emit valid x86 assembly instructions, structuring data definitions and operational code blocks correctly:

```
.data
a DW 0
b DW 0
c DW 0
d DW 0
```

```
MOV AX, 9
MOV temp0, AX
MOV AX, temp0
MOV a, AX
MOV AX, a
ADD AX, 5
MOV temp1, AX
MOV AX, temp1
MOV b, AX
MOV AX, 4
MOV temp2, AX
MOV AX, temp2
MOV c, AX
...
HLT
```

5.9 Discussion

Evaluating the compilation process reveals how the individual phases interact within the toolchain. The Flex lexer correctly isolates string literals and maps patterns to tokens. The Bison parser resolves expression operations according to declared operator precedence rules, avoiding ambiguous evaluation paths.

The symbol table tracks values reliably, allowing the system to perform compile-time constant folding on pure literal operations. This optimization pass helps minimize redundant execution steps in the final output phase. The resulting assembly code maps cleanly to standard hardware instructions, demonstrating a reliable translation pipeline from source strings down to low-level instructions.

CHAPTER 6: CONCLUSION AND FUTURE WORK

6.1 Summary

This project successfully designed and built a working multi-pass Mini Compiler framework on an Ubuntu Linux system, leveraging GNU Flex for lexical scanning and GNU Bison for syntax parsing. The toolchain handles character streaming, token isolation, syntax validation, variable state tracking, intermediate code generation, and constant folding optimizations, ultimately producing valid x86-64 assembly instructions.

6.2 Achievements

The compiler implementation achieved several distinct design goals:

- Built a deterministic lexical scanner capable of filtering comments, tracking line coordinates, and isolating strings.
- Designed an operator-precedence grammar that processes expression operations without shift-reduce structural conflicts.
- Implemented a reverse-scanning symbol table that manages variable states and detects undeclared identifiers.
- Created an intermediate three-address code generator based on a quadruples array design.
- Developed an optimization pass that executes constant folding on literal numbers at compile time.
- Engineered a backend emitter that outputs clean x86 assembly representations.

6.3 Limitations

While the current compiler functions correctly as an end-to-end translation pipeline, it operates under a few design simplifications:

- **Single Data Type:** It only supports standard signed integers, lacking native features for floats, booleans, or character arrays.
- **Simple Scope Model:** The system uses a flat global storage structure, meaning it lacks deep nested scope support for blocks or functions.
- **Basic Control Flow:** The lexer extracts keywords like `if` and `while`, but the parser rules do not generate the jump and label instructions required to execute conditional loops or logical branches.
- **Simple Optimization:** The optimization phase is limited to local constant folding, lacking broader techniques like dead-code elimination or global constant propagation.

6.4 Future Enhancements

The compiler framework can be expanded with several architectural additions:

- **Array and Type Support:** Adding float and character array tracking to the lexer and parser.
- **Control Flow Generation:** Extending the TAC engine to generate relational jump instructions and labels for conditional statements and loops.
- **Advanced Optimization Passes:** Adding register allocation models and dead-code elimination routines to optimize memory operations.
- **Function Call Infrastructure:** Engineering a runtime stack architecture to support nested function calls and local stack frames.

6.5 Conclusion

This project demonstrates the practical application of compiler design principles, successfully transforming theoretical language rules into working software. Using Flex and Bison to manage scanning and parsing simplified the front-end development, allowing more focus on the symbol table mechanics, optimization routines, and assembly code emission. The resulting multi-pass toolchain provides a clear look at how high-level code scales down to low-level hardware instructions, serving as a solid foundation for more advanced language engineering tasks.

Github Link: https://github.com/raphaelmustafi/MINI_COMPILER